

ITA0605 - Machine Learning

List of Lab Exercises

6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

Aim:

To implement the **Naïve Bayes algorithm** using the Iris dataset and evaluate its performance using the **confusion matrix** and **accuracy**.

Algorithm:

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Create a Gaussian Naïve Bayes classifier.
5. Train the model using the training data.
6. Predict the class labels for the test data.
7. Display the confusion matrix.
8. Calculate and display the accuracy

Dataset:

Program Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score,
ConfusionMatrixDisplay

import matplotlib.pyplot as plt

iris = load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3, random_state=1)
```

```
model = GaussianNB() model.fit(X_train, y_train) y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred) print("Confusion Matrix:")
print(cm)
```

```
acc = accuracy_score(y_test, y_pred) print("\nAccuracy:", acc)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
```

```
display_labels=iris.target_names)
```

```
disp.plot(cmap='Blues') plt.title("Confusion Matrix") plt.show()
```

Output:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

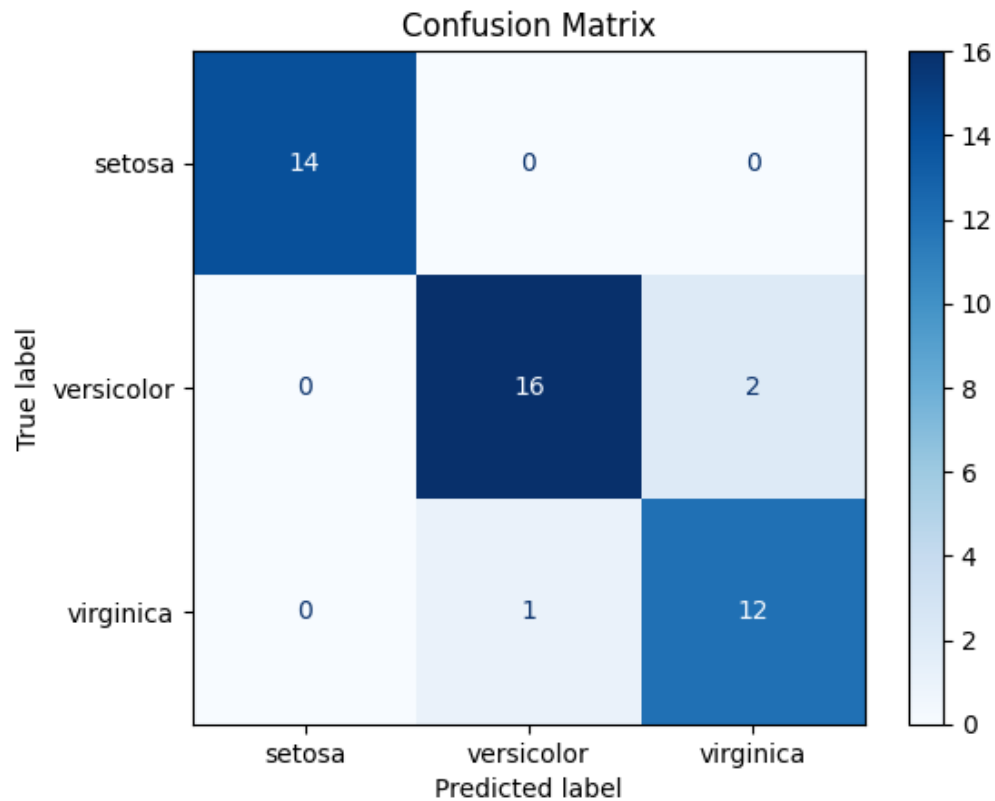
acc = accuracy_score(y_test, y_pred)
print("\nAccuracy:", acc)

disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=iris.target_names)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

*** Confusion Matrix:

```
[[14  0  0]
 [ 0 16  2]
 [ 0  1 12]]
```

Accuracy: 0.9333333333333333



Dataset 2:

Program Code:

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
data = [
    [8, 780, 5, 'Yes'],
    [7, 760, 4, 'Yes'],
    [6, 730, 5, 'Yes'],
    [4, 620, 8, 'No'],
    [3, 590, 9, 'No'],
    [9, 810, 4, 'Yes'],
    [5, 650, 7, 'No'],
    [8, 770, 5, 'Yes'],
    [4, 610, 8, 'No'],
    [7, 750, 6, 'Yes']
]
```

```
columns = [
    'Income',
```

```

    'Credit Score',
    'Loan Amount',
    'Loan Approved'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)

X = df[
    ['Income', 'Credit Score', 'Loan Amount']
]

y = df['Loan Approved']

y_encoded = y.map({
    'No': 0,
    'Yes': 1
})

model = GaussianNB()

model.fit(X, y_encoded)

y_pred = model.predict(X)

cm = confusion_matrix(y_encoded, y_pred)

print("\nConfusion Matrix:")
print(cm)

accuracy = accuracy_score(y_encoded, y_pred)

print("\nAccuracy:")
print(accuracy)

print("\nAccuracy Percentage:")
print(accuracy * 100, "%")

result = pd.DataFrame({
    'Actual': y,
    'Predicted': [
        'Yes' if value == 1 else 'No'
        for value in y_pred
    ]
})

print("\nActual vs Predicted:")
print(result)

```

Output:

```
Original Dataset:
***  Income  Credit Score  Loan Amount  Loan Approved
0      8      780      5      Yes
1      7      760      4      Yes
2      6      730      5      Yes
3      4      620      8      No
4      3      590      9      No
5      9      810      4      Yes
6      5      650      7      No
7      8      770      5      Yes
8      4      610      8      No
9      7      750      6      Yes

Naive Bayes Model Training Completed Successfully.

Confusion Matrix:
[[4 0]
 [0 6]]

Accuracy:
1.0

Accuracy Percentage:
100.0 %

Actual vs Predicted:
Actual Predicted
0  Yes      Yes
1  Yes      Yes
2  Yes      Yes
3  No       No
4  No       No
5  Yes      Yes
6  No       No
7  Yes      Yes
8  No       No
```

Dataset 3:

Program Code:

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

data = [
    [39.1, 112, 94, 'Positive'],
    [38.8, 109, 95, 'Positive'],
    [37.0, 78, 99, 'Negative'],
    [39.3, 115, 93, 'Positive'],
    [36.8, 74, 98, 'Negative'],
    [38.7, 110, 94, 'Positive'],
    [37.1, 80, 98, 'Negative'],
    [39.0, 113, 92, 'Positive'],
```

```

    [36.9, 76, 99, 'Negative'],
    [38.9, 111, 93, 'Positive']
]

columns = [
    'Temperature',
    'Heart Rate',
    'Oxygen Level',
    'Disease'
]

df = pd.DataFrame(data, columns=columns)

print("Original Dataset:")
print(df)

X = df[
    ['Temperature', 'Heart Rate', 'Oxygen Level']
]

y = df['Disease']

y_encoded = y.map({
    'Negative': 0,
    'Positive': 1
})

model = GaussianNB()

model.fit(X, y_encoded)

print("\nNaive Bayes Model Training Completed Successfully.")

y_pred = model.predict(X)

cm = confusion_matrix(y_encoded, y_pred)

print("\nConfusion Matrix:")
print(cm)

accuracy = accuracy_score(y_encoded, y_pred)

print("\nAccuracy:")
print(accuracy)

print("\nAccuracy Percentage:")

```

```

print(accuracy * 100, "%")

result = pd.DataFrame({
    'Actual': y,
    'Predicted': [
        'Positive' if value == 1 else 'Negative'
        for value in y_pred
    ]
})

print("\nActual vs Predicted:")
print(result)

```

Output:

```

Original Dataset:
...  Temperature  Heart Rate  Oxygen Level  Disease
0      39.1      112      94  Positive
1      38.8      109      95  Positive
2      37.0       78      99  Negative
3      39.3      115      93  Positive
4      36.8       74      98  Negative
5      38.7      110      94  Positive
6      37.1       80      98  Negative
7      39.0      113      92  Positive
8      36.9       76      99  Negative
9      38.9      111      93  Positive

Naive Bayes Model Training Completed Successfully.

Confusion Matrix:
[[4 0]
 [0 6]]

Accuracy:
1.0

Accuracy Percentage:
100.0 %

Actual vs Predicted:
  Actual Predicted
0  Positive  Positive
1  Positive  Positive
2  Negative  Negative
3  Positive  Positive
4  Negative  Negative
5  Positive  Positive
6  Negative  Negative
7  Positive  Positive
8  Negative  Negative

```

Dataset 4:

Program Code:

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
data = [
    [10, 95, 50, 'Yes'],
    [9, 92, 48, 'Yes'],
    [8, 90, 45, 'Yes'],
    [4, 70, 25, 'No'],
    [3, 65, 20, 'No'],
    [11, 96, 52, 'Yes'],
    [5, 75, 30, 'No'],
    [8, 89, 44, 'Yes'],
    [2, 60, 18, 'No'],
    [7, 87, 40, 'Yes']
]
```

```
columns = [
    'Experience',
    'Performance Score',
    'Training Hours',
    'Promotion'
]
```

```
df = pd.DataFrame(data, columns=columns)
```

```
print("Original Dataset:")
print(df)
```

```
X = df[
    ['Experience', 'Performance Score', 'Training Hours']
]
```

```
y = df['Promotion']
```

```
y_encoded = y.map({
    'No': 0,
    'Yes': 1
})
```



```
model = GaussianNB()

model.fit(X, y_encoded)

print("\nNaive Bayes Model Training Completed Successfully.")

y_pred = model.predict(X)

cm = confusion_matrix(y_encoded, y_pred)

print("\nConfusion Matrix:")
print(cm)

accuracy = accuracy_score(y_encoded, y_pred)

print("\nAccuracy:")
print(accuracy)

print("\nAccuracy Percentage:")
print(accuracy * 100, "%")

result = pd.DataFrame({
    'Actual': y,
    'Predicted': [
        'Yes' if value == 1 else 'No'
        for value in y_pred
    ]
})

print("\nActual vs Predicted:")
print(result)
```

Output:

```
Original Dataset:
---  Experience  Performance Score  Training Hours  Promotion
0      10      95      50      Yes
1       9      92      48      Yes
2       8      90      45      Yes
3       4      70      25      No
4       3      65      20      No
5      11      96      52      Yes
6       5      75      30      No
7       8      89      44      Yes
8       2      60      18      No
9       7      87      40      Yes
```

Naive Bayes Model Training Completed Successfully.

Confusion Matrix:

```
[[4 0]
 [0 6]]
```

Accuracy:

1.0

Accuracy Percentage:

100.0 %

Actual vs Predicted:

	Actual	Predicted
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	No	No
4	No	No
5	Yes	Yes
6	No	No
7	Yes	Yes
8	No	No

Dataset 5:

Program Code:

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
data = [
    [150, 7.5, 90, 'Apple'],
    [160, 7.8, 88, 'Apple'],
    [145, 7.3, 91, 'Apple'],
    [120, 5.5, 70, 'Orange'],
    [125, 5.8, 72, 'Orange'],
    [130, 6.0, 74, 'Orange'],
    [155, 7.6, 89, 'Apple'],
    [118, 5.4, 69, 'Orange'],
    [148, 7.4, 92, 'Apple'],
    [122, 5.7, 71, 'Orange']
]
```

```
]
```

```
columns = [  
    'Weight',  
    'Diameter',  
    'Sweetness',  
    'Fruit'  
]
```

```
df = pd.DataFrame(data, columns=columns)
```

```
print("Original Dataset:")  
print(df)
```

```
X = df[  
    ['Weight', 'Diameter', 'Sweetness']  
]
```

```
y = df['Fruit']
```

```
y_encoded = y.map({  
    'Orange': 0,  
    'Apple': 1  
})
```

```
model = GaussianNB()
```

```
model.fit(X, y_encoded)
```

```
print("\nNaive Bayes Model Training Completed Successfully.")
```

```
y_pred = model.predict(X)
```

```
cm = confusion_matrix(y_encoded, y_pred)
```

```
print("\nConfusion Matrix:")  
print(cm)
```

```
accuracy = accuracy_score(y_encoded, y_pred)
```

```
print("\nAccuracy:")  
print(accuracy)
```

```
print("\nAccuracy Percentage:")  
print(accuracy * 100, "%")
```

```

result = pd.DataFrame({
    'Actual': y,
    'Predicted': [
        'Apple' if value == 1 else 'Orange'
        for value in y_pred
    ]
})

print("\nActual vs Predicted:")
print(result)

```

Output:

```

Original Dataset:
...  Weight  Diameter  Sweetness  Fruit
0      150      7.5      90  Apple
1      160      7.8      88  Apple
2      145      7.3      91  Apple
3      120      5.5      70  Orange
4      125      5.8      72  Orange
5      130      6.0      74  Orange
6      155      7.6      89  Apple
7      118      5.4      69  Orange
8      148      7.4      92  Apple
9      122      5.7      71  Orange

Naive Bayes Model Training Completed Successfully.

Confusion Matrix:
[[5 0]
 [0 5]]

Accuracy:
1.0

Accuracy Percentage:
100.0 %

Actual vs Predicted:
Actual Predicted
0  Apple  Apple
1  Apple  Apple
2  Apple  Apple
3  Orange  Orange
4  Orange  Orange
5  Orange  Orange
6  Apple  Apple
7  Orange  Orange
8  Apple  Apple

```

Result:

The **Naïve Bayes classifier** was implemented successfully. The model classified the Iris dataset with high accuracy, and the confusion matrix showed the classification performance.

